

Step 1 — The Two-Agent Pipeline, Running Standalone — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (a real, typed version), **Advanced** (the part that actually matters for a loop like this). Read Basic first even if you already know the pattern — it's the fastest way to see exactly what each deeper level adds.

- [Hint 1 — Researcher and Fact-Checker, each on their own](#)
- [Hint 2 — The loop, and the hard cap](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Researcher and Fact-Checker, Each on Their Own

Basic Version

You need two small, separate pieces before any loop exists:

- Something that takes a topic (and maybe some earlier feedback) and writes a short summary.
- Something that reads a summary and says whether it's good enough, and if not, why.

Build and test each one by itself first, with a plain `print()` in a scratch script, before wiring them together.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The Researcher is `run_researcher(topic: str, feedback: str | None = None) -> Draft`. `Draft` is just a small data shape — a `pydantic.BaseModel` with `topic: str` and `summary: str` works well, the same idea Project 4 used for its own agent outputs. When `feedback` is `None`, this is the first draft. When it's given, the prompt has to say something like "fix exactly the issues below, don't rewrite parts that weren't flagged" — otherwise the model tends to rewrite the whole thing from scratch every round, which makes it hard to tell if your loop is actually converging or just generating a new random draft each time.

The Fact-Checker is `run_fact_checker(draft: Draft) -> FactCheckVerdict`. `FactCheckVerdict` needs `approved: bool` and `issues: list[str]`. The cleanest way to get a real Python object back instead of parsing free text yourself is `ChatOpenAI(...).with_structured_output(FactCheckVerdict)` — LangChain handles turning the model's response into that exact shape for you, so you don't have to write your own "look for the word APPROVED" text parser.

Pieces to use: - `from langchain_openai import ChatOpenAI` - `from langchain_core.prompts import ChatPromptTemplate` - `from pydantic import BaseModel` - `.with_structured_output(YourPydanticModel)` on the Fact-Checker's `ChatOpenAI` instance

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The real risk here isn't the code — it's writing a Fact-Checker prompt so soft it approves everything on the first try. If that happens, your loop "works" in the sense that it runs without crashing, but you've never actually tested the part that matters: a second round that genuinely fixes something the first round got wrong.

Write the Fact-Checker's scoring criteria into its prompt explicitly, the same lesson Project 4's Reviewer prompt taught: name the actual categories of problem you want it to catch — a wrong date, a wrong name, an invented fact, a claim stated with more confidence than the evidence supports. A model told only "check if this is good" will say yes almost every time; a model told exactly what "bad" looks like will actually find it.

Also think about what happens to `feedback` between rounds. If you only ever pass the *most recent* round's issues, and the Researcher's fix for Round 1's problem accidentally reintroduces something Round 0 already got right, nothing in your current design would catch that — that's a real, known limitation of this simple version, not something you need to solve now, but worth noticing on purpose rather than being surprised by later.

Difference between Basic, Intermediate, and Advanced: Basic describes the two pieces you need and how to sanity-check them. Intermediate gives the real function signatures, the real data shapes, and the actual LangChain tool (`with_structured_output`) that gets you a typed verdict instead of a hand-written text parser. Advanced is about the one mistake that quietly ruins this step — a Fact-Checker that approves everything — and why writing real criteria into its prompt is the difference between a loop that's actually been tested and one that just hasn't failed yet.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The Loop, and the Hard Cap

Basic Version

```
loop up to max_rounds times:
    draft = ask the Researcher (using last round's feedback, if any)
    verdict = ask the Fact-Checker about that draft
    remember what happened this round

    if the Fact-Checker approved it:
        stop the loop

    otherwise:
        feedback = the Fact-Checker's issues, for next time

return the last draft, plus whether it was ever approved
```

The important part: the loop stops after `max_rounds` no matter what. It never keeps going "just one more try."

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

A plain Python `for` loop with a `break` does this cleanly — you don't need a `while True` with manual counting:

```
for round_number in range(1, max_rounds + 1):
    draft = run_researcher(topic, feedback=feedback)
    verdict = run_fact_checker(draft)

    # ...record round_number, draft, verdict somewhere...

    if verdict.approved:
        break
    feedback = "; ".join(verdict.issues)
```

`range(1, max_rounds + 1)` gives you round numbers `1, 2, 3` for `max_rounds=3` — human-readable round numbers, not zero-indexed ones. `"; ".join(verdict.issues)` turns a list of issue strings into one feedback string simply; a fancier version could number them, but a plain join is honest and readable.

Collect each round's result in a list as you go — something like a small `RevisionRound` model with `round_number`, `summary`, `approved`, and `issues` — so `main.py` (and later, the MCP layer) has the full history, not just the final answer.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think carefully about what the function returns when the cap is hit *without* approval — this is Doc11's non-convergence lesson, applied here. It should not be an exception, and it should not silently pretend everything's fine. `PipelineResult` needs an `approved: bool` field the caller can actually check, separate from `final_summary` — a caller that only looks at the summary text has no reliable way to tell "this is a confirmed, checked answer" from "this is just whatever we had left when we ran out of tries."

Also decide, on purpose, what "the last draft" means when the loop ends unapproved. In the loop above, `draft` and `verdict` are whatever the *final* iteration produced — which is correct (it's the most-revised version you have), but it's easy to accidentally return an earlier round's draft if you reorganize the loop later. A quick sanity check: after the loop, `rounds[-1].summary` should always equal `draft.summary`. If it doesn't, something in your loop is tracking the wrong round.

One more thing worth testing now, even though Step 4 is where you really handle it: what happens today if `run_researcher` or `run_fact_checker` raises a real exception (a network error, a malformed API response)? Right now, nothing catches it — it just crashes `main.py`. That's fine for Step 1. Just don't be surprised by it, and don't try to fix it yet; Step 4 is where that gets handled properly, with the full picture of what "properly" needs to mean for an MCP tool.

Difference between Basic, Intermediate, and Advanced: Basic is the plain loop shape and the one rule that matters (it always stops). Intermediate is the real Python — a `for / break` loop, the exact join for building feedback text, and the shape of what to record each round. Advanced is about the return value being honest: a caller must be able to tell "approved" from "ran out of tries" without guessing, and about noticing — without yet fixing — the unhandled-exception gap that Step 4 exists to close.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)